From Linear Scores to a Single Hidden Layer:

A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project — National AI Center — AI Academy

Nihat Rzayev Fidan Allahverdiyeva Ibrahim Mammadov Fatma Mammadova

Abstract

This report investigates when a one-hidden-layer neural network with tanh activations genuinely outperforms a linear softmax classifier, and when additional model complexity is unnecessary. We compare both models on three datasets: a linearly separable Gaussian task, a nonlinearly separable moons task, and the ten-class Digits benchmark. On the linear Gaussian task, softmax regression achieves near-perfect separation and the neural network provides no measurable benefit. On the moons task, only a hidden layer of width 32 consistently separates the classes (test accuracy 0.975); widths of 2 and 8 achieve only 0.850, matching the linear baseline. On the Digits benchmark, softmax regression reaches a mean test accuracy of 0.935 ± 0.006 (95% CI, 5 seeds), while the neural network (Adam, hidden width 32) reaches 0.954 ± 0.005. Advanced Track B reveals that the neural network produces better-calibrated confidence scores. Our central finding is that representational complexity is justified only when the task geometry demands it, and that repeated-seed evaluation is essential for distinguishing genuine improvement from initialization luck.

Contents

1 Introduction **3**

2 Background and Mathematical Setup **4**
2.1 Notation . . . 4
2.2 Supervised Classification . . . 4
2.3 Softmax Regression as a Probabilistic Model . . . 4
2.4 Cross-Entropy Loss as Negative Log-Likelihood . . . 5
2.5 Linear Decision Boundaries . . . 5
2.6 The One-Hidden-Layer Network . . . 5
2.7 Why the Gaussian Task Suits a Linear Model . . . 6
2.8 Why the Moons Task Requires Nonlinearity . . . 6

3 Methods **6**
3.1 Datasets . . . 6
3.2 Preprocessing . . . 6
3.3 Softmax Regression (Model 1) . . . 6
3.4 One-Hidden-Layer Neural Network (Model 2) . . . 7
3.5 Optimizers . . . 7
3.6 Training Protocol and Hyperparameters . . . 8
3.7 Evaluation Metrics . . . 8

4 Implementation Sanity Checks **8**

5 Experiments **9**
5.1 Core Comparison 1: Linear Gaussian Task 9
5.2 Core Comparison 2: Moons Task 10
5.3 Capacity Ablation on Moons 11
5.4 Core Comparison 3: Digits Benchmark 12
5.5 Optimizer Study on Digits 14
5.6 Failure-Case Analysis 15

6 Advanced Track B: Prediction Confidence and Reliability **16**
6.1 Definitions 16
6.2 Reliability Diagrams 16
6.3 Confidence on Correct vs. Incorrect Predictions 17

7 Discussion **17**

8 Limitations **18**

1 Introduction

A central tension in machine learning is the trade-off between model simplicity and representational power. Simple models are easier to analyse, less prone to overfitting, and more interpretable; complex models may capture richer structure in data, but at the cost of additional parameters, longer training, and a greater risk of fitting noise rather than signal.

This capstone asks one scientifically precise question:

When does a one-hidden-layer nonlinear classifier genuinely improve on a linear decision rule, and when is additional model complexity unnecessary?

The two models we compare are **softmax regression**—a linear multiclass classifier—and a **one-hidden-layer neural network** with tanh activations and a softmax output layer. Both are implemented from scratch in NumPy; no automatic-differentiation or machine-learning library is used for the core computations.

The comparison is scientifically meaningful because the two model classes differ in exactly one structural respect: the presence or absence of a single nonlinear hidden representation. This controlled difference allows any observed performance gap to be attributed directly to representational capacity.

We answer the central question through:

- three datasets (Gaussian blobs, moons, and Digits);
- three required ablations (capacity on moons, optimizer study on Digits, and a deliberate failure-case analysis);
- repeated-seed statistics with 95% confidence intervals; and
- Advanced Track B: prediction confidence and reliability.

2 Background and Mathematical Setup

2.1 Notation

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^d$	Input feature vector for one example
$y \in \{0, \ldots, k-1\}$	True integer class label
n	Batch size
d	Input dimensionality ($d = 64$ for Digits)
k	Number of output classes ($k = 10$ for Digits)
m	Hidden layer width
$\mathbf{W}_1 \in \mathbb{R}^{m \times d},\ \mathbf{b}_1 \in \mathbb{R}^{1 \times m}$	Hidden-layer weight matrix and bias
$\mathbf{W}_2 \in \mathbb{R}^{k \times m},\ \mathbf{b}_2 \in \mathbb{R}^{1 \times k}$	Output-layer weight matrix and bias
$\mathbf{Z}_1 \in \mathbb{R}^{n \times m}$	Hidden pre-activations (batch)
$\mathbf{H} \in \mathbb{R}^{n \times m}$	Hidden activations after tanh (batch)
$\mathbf{S} \in \mathbb{R}^{n \times k}$	Output logits (batch)
$\mathbf{P} \in \mathbb{R}^{n \times k}$	Predicted class probabilities (batch)
$\mathbf{Y} \in \{0, 1\}^{n \times k}$	One-hot label matrix (batch)
$\mathcal{L}$	Scalar loss value
λ	L2 regularization coefficient

2.2 Supervised Classification

In supervised classification we observe input-output pairs $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$. Each $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is a feature vector and $y^{(i)} \in \{0, \ldots, k-1\}$ is the true class label. All comparisons use the same fixed train/validation/test partition for fairness.

2.3 Softmax Regression as a Probabilistic Model

Softmax regression computes raw *logit scores*

$$\mathbf{s}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{k \times d},\ \mathbf{b} \in \mathbb{R}^{1 \times k},$$

and converts them to a valid probability vector via the softmax map:

$$p_j(\mathbf{x}) = \frac{\exp(s_j(\mathbf{x}))}{\sum_{\ell=0}^{k-1} \exp(s_\ell(\mathbf{x}))}, \qquad j = 0, \ldots, k-1.$$

This defines the conditional probability model $P(Y = j \mid \mathbf{X} = \mathbf{x}; \mathbf{W}, \mathbf{b}) = p_j(\mathbf{x})$. The entries of $\mathbf{W}\mathbf{x} + \mathbf{b}$ are unnormalized log-probability scores; softmax is the normalization map that converts them into a valid categorical distribution [2, 4].

Numerical stability. Our implementation subtracts $\max_j s_j(\mathbf{x})$ before exponentiation:

$$p_j(\mathbf{x}) = \frac{\exp(s_j - s_{\max})}{\sum_\ell \exp(s_\ell - s_{\max})}.$$

This leaves the probabilities unchanged but prevents numerical overflow.

2.4 Cross-Entropy Loss as Negative Log-Likelihood

For a single example with true class y, the cross-entropy loss is $\mathcal{L}(\mathbf{x}, y) = -\log p_y(\mathbf{x})$.

Derivation. If we treat the observed label $y^{(i)}$ as a draw from the model's categorical distribution, the likelihood of observing $y^{(i)}$ is $p_{y^{(i)}}(\mathbf{x}^{(i)})$. Taking the log and negating, then averaging over N examples:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{y^{(i)}}(\mathbf{x}^{(i)}).$$

Minimizing this is *exactly* equivalent to maximizing the average log-likelihood of the observed labels under the model [2, 3]. In vectorized form, using one-hot labels $\mathbf{Y} \in \{0,1\}^{n\times k}$:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{i,j} Y_{ij}\log(P_{ij} + \varepsilon), \qquad \varepsilon = 10^{-15}.$$

L2 regularization adds a penalty $\frac{\lambda}{2}(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$ to prevent weight explosion.

Worked example. A three-class model produces scores $\mathbf{s} = [1.2,\ 0.2,\ -0.4]$. Then $\exp(\mathbf{s}) \approx [3.32, 1.22, 0.67]$ and softmax gives $\mathbf{p} \approx [0.64, 0.23, 0.13]$. If the true class is class 0, the loss is $-\log(0.64) \approx 0.45$. If it were class 1, the loss would be $-\log(0.23) \approx 1.47$—much larger because the model assigns less probability to the correct label.

2.5 Linear Decision Boundaries

For the softmax model, the decision boundary between classes j and ℓ satisfies $s_j(\mathbf{x}) = s_\ell(\mathbf{x})$, which gives

$$(\mathbf{w}_j - \mathbf{w}_\ell)^\top \mathbf{x} + (b_j - b_\ell) = 0.$$

In two dimensions this is a line; in d dimensions it is a hyperplane. Softmax regression can therefore only learn *linear* decision boundaries, and any dataset whose classes cannot be separated by hyperplanes is structurally beyond its representational capacity.

2.6 The One-Hidden-Layer Network

The neural network introduces a single hidden layer with m units and tanh activations:

$$\mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2.$$

The output scores $\mathbf{s}$ are converted to probabilities via the same softmax map. Because $\mathbf{h}$ is a *nonlinear* function of $\mathbf{x}$, the decision boundary in the original input space is generally curved.

Why tanh**.** The hyperbolic tangent is smooth, zero-centred, and its derivative has the compact closed form $\tanh'(z) = 1 - \tanh^2(z)$, which makes the backpropagation derivation clean and interpretable [3].

2.7 Why the Gaussian Task Suits a Linear Model

In the linear Gaussian task each class is generated from a Gaussian with the same covariance matrix and different means. By a classical result, the Bayes-optimal boundary under equal-covariance Gaussians is a *linear* function of the input (the LDA boundary). Softmax regression's hypothesis class includes all linear boundaries, so it can in principle recover the optimal rule. The hidden layer can represent the same linear boundary as a special case, so it cannot do worse, but it has no structural advantage.

2.8 Why the Moons Task Requires Nonlinearity

The moons dataset places two interleaved crescent-shaped clusters that are not linearly separable: no hyperplane correctly classifies all examples. A linear classifier must incur non-trivial error regardless of how its weights are set. A network with hidden units can compose affine maps with tanh nonlinearities to produce curved boundaries, making it geometrically capable of separating the crescents. The representational gap is therefore *structural*.

3 Methods

3.1 Datasets

Linear Gaussian task. Two-dimensional, two-class dataset generated as two Gaussian blobs with mild overlap.

Moons task. Two-dimensional, two-class crescent dataset. No linear classifier achieves near-perfect accuracy.

Digits benchmark. Scikit-learn Digits: 1 797 greyscale images of handwritten digits 0–9, each stored as a flattened 64-dimensional vector with pixel values already scaled to $[0, 1]$. The fixed train/validation/test split from `digits_split_indices.npz` is used universally; no additional normalization or augmentation is applied (Protocol §4.2).

3.2 Preprocessing

Synthetic tasks. Features are standardized (zero mean, unit variance) using statistics computed only on the training split. Validation and test features are transformed with the same statistics to prevent data leakage.

Digits benchmark. Features are already in $[0, 1]$; no further preprocessing is applied. One-hot encoding is applied to integer labels to produce the matrix $\mathbf{Y}$.

3.3 Softmax Regression (Model 1)

Forward pass.

$$\mathbf{S} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}\mathbf{b}, \qquad P_{ij} = \frac{\exp(S_{ij})}{\sum_\ell \exp(S_{i\ell})}.$$

Loss.

$$\mathcal{L} = -\frac{1}{n}\sum_{i,j} Y_{ij}\log(P_{ij}+\varepsilon) + \frac{\lambda}{2}\|\mathbf{W}\|_F^2.$$

Gradient.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{S}} = \frac{1}{n}(\mathbf{P}-\mathbf{Y}),$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{X} + \lambda \mathbf{W}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{1}.$$

3.4 One-Hidden-Layer Neural Network (Model 2)

Forward pass (batch).

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{1}\mathbf{b}_1 \in \mathbb{R}^{n\times m}, \tag{1}$$
$$\mathbf{H} = \tanh(\mathbf{Z}_1) \in \mathbb{R}^{n\times m}, \tag{2}$$
$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{1}\mathbf{b}_2 \in \mathbb{R}^{n\times k}, \tag{3}$$
$$\mathbf{P} = \text{softmax_row}(\mathbf{S}) \in \mathbb{R}^{n\times k}. \tag{4}$$

Complete backpropagation derivation. Starting from $\mathcal{L} = -\frac{1}{n}\operatorname{tr}(\mathbf{Y}^\top \log \mathbf{P}) + \frac{\lambda}{2}(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$:

Step 1. Sensitivity at the output logits (standard softmax + CE result):

$$\delta^S \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{S}} = \frac{1}{n}(\mathbf{P}-\mathbf{Y}).$$

Step 2. Gradients for output-layer parameters:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = (\delta^S)^\top \mathbf{H} + \lambda \mathbf{W}_2, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = (\delta^S)^\top \mathbf{1}.$$

Step 3. Propagate error through $\mathbf{W}_2$ to the hidden layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{H}} = \delta^S \mathbf{W}_2.$$

Step 4. Pass through the tanh activation using $\tanh'(z) = 1 - \tanh^2(z)$:

$$\delta^{Z_1} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{H}} \odot (1 - \mathbf{H}\odot\mathbf{H}).$$

Step 5. Gradients for hidden-layer parameters:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = (\delta^{Z_1})^\top \mathbf{X} + \lambda \mathbf{W}_1, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = (\delta^{Z_1})^\top \mathbf{1}.$$

These five steps completely specify the gradient computation via the chain rule; no external tools are used.

3.5 Optimizers

Let θ denote any parameter and g_t its gradient at step t.

SGD. $\theta \leftarrow \theta - \eta\, g_t$.

Momentum. $v_t = \beta v_{t-1} - \eta g_t, \quad \theta \leftarrow \theta + v_t$.

Adam.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta \leftarrow \theta - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

3.6 Training Protocol and Hyperparameters

Setting	Value
Epoch budget	200
Mini-batch size	64
L2 regularization λ	10^{-4}
Default hidden width (Digits)	32
Softmax baseline	SGD, $\eta = 0.05$
NN SGD	$\eta = 0.05$
NN Momentum	$\eta = 0.05$, $\beta = 0.9$
NN Adam	$\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$
Checkpoint policy (Digits)	Best validation cross-entropy within budget
Repeated seeds (NN)	$\{11, 22, 33, 44, 55\}$
Repeated seeds (Softmax)	$\{0, 1, 2, 3, 4\}$

3.7 Evaluation Metrics

Accuracy. $\mathrm{Acc} = N^{-1} \sum_i \mathbf{1}[\hat{y}^{(i)} = y^{(i)}]$.

Mean cross-entropy (CE). $\mathrm{CE} = -N^{-1} \sum_{i,j} Y_{ij} \log(P_{ij} + \varepsilon)$.

95% confidence interval (5 seeds). $\bar{x} \pm 2.776 \cdot s/\sqrt{5}$, where s is the sample standard deviation.

4 Implementation Sanity Checks

Before running any experiment we verified correctness through five concrete checks. All numerical values below come directly from `sanity_checks_nn.json`.

Check 1: Probability sums. Computed $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$ on a random batch. Observed: min row sum $= 0.9999999999999999$, max row sum $= 1.0000000000000002$. All row sums lie within 10^{-14} of one. **Passed.**

Check 2: Loss decreases on a tiny subset. Ran 50 gradient-descent steps on 8 training examples. Observed: initial loss = 2.3025, final loss = 2.1491. The loss decreased monotonically, confirming that forward pass, loss computation, and backward pass are mutually consistent. **Passed.**

Check 3: Tiny-subset overfitting. Ran 500 epochs on 10 training examples. Observed: best epoch = 500, training accuracy = 1.000. This confirms that the network has sufficient capacity to overfit small data, ruling out silent bugs in the weight update. **Passed.**

Check 4: No NaN or Inf during training. Instrumented the training loop to assert finiteness after every parameter update. Observed: `has_nan = false`, `has_inf = false`. **Passed.**

Check 5: Finite-difference gradient verification. Compared analytically computed gradients with central finite differences ($h = 10^{-5}$) on selected entries of $\mathbf{W}_1$ and $\mathbf{W}_2$:

Param	Index	Analytic	Numeric	Rel. Error
$\mathbf{W}_1$	$(0, 0)$	3.047×10^{-7}	3.047×10^{-7}	8.6×10^{-6}
$\mathbf{W}_1$	$(3, 5)$	-1.490×10^{-4}	-1.490×10^{-4}	3.7×10^{-8}
$\mathbf{W}_2$	$(0, 0)$	3.450×10^{-4}	3.450×10^{-4}	4.4×10^{-8}
$\mathbf{W}_2$	$(2, 4)$	1.045×10^{-3}	1.045×10^{-3}	1.5×10^{-9}

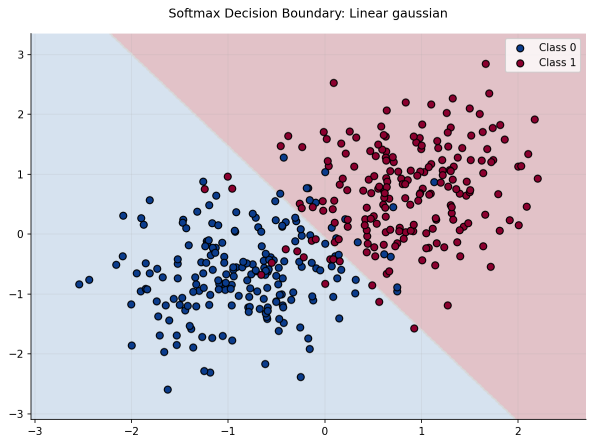
All relative errors are below 10^{-5}, confirming the backpropagation derivation is correct. **Passed.**

5 Experiments

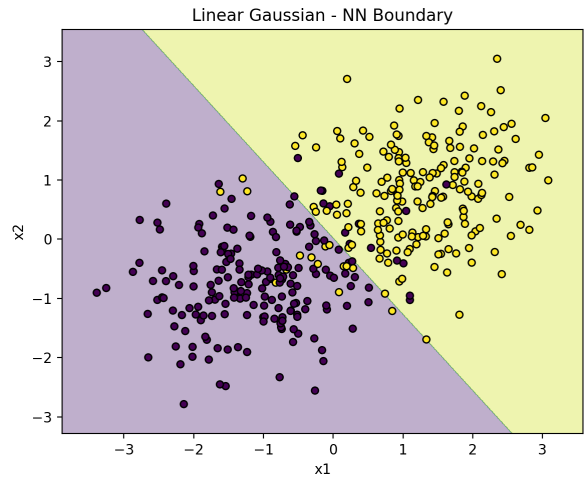
5.1 Core Comparison 1: Linear Gaussian Task

Setup. Both models are trained for 200 epochs. The final-epoch checkpoint is reported, since the dataset is small and overfitting is negligible.

Results. Figure 1 shows the decision boundaries. Both models learn a nearly optimal linear separator.

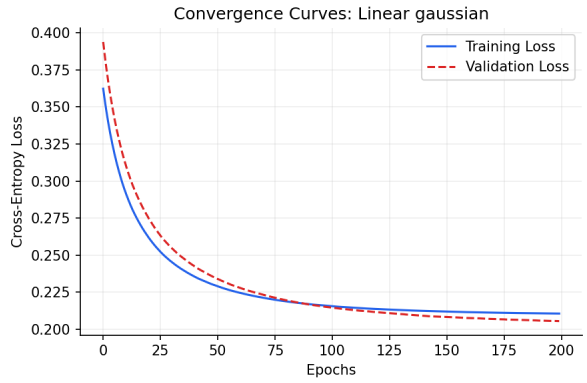


(a) Softmax: linear boundary.

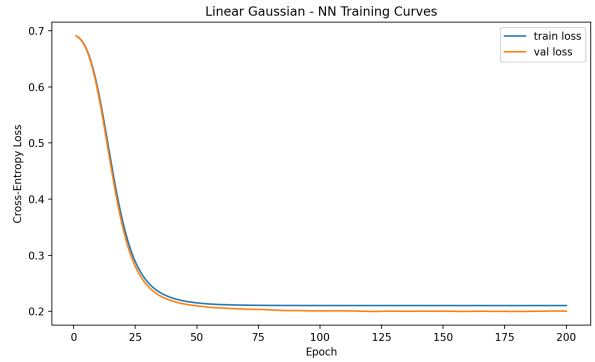


(b) NN: also learns a near-linear boundary.

Figure 1: Decision boundaries on the Linear Gaussian task. Both models find near-optimal linear separators; the hidden layer provides no meaningful benefit.



(a) Softmax convergence curves.



(b) Neural network convergence curves.

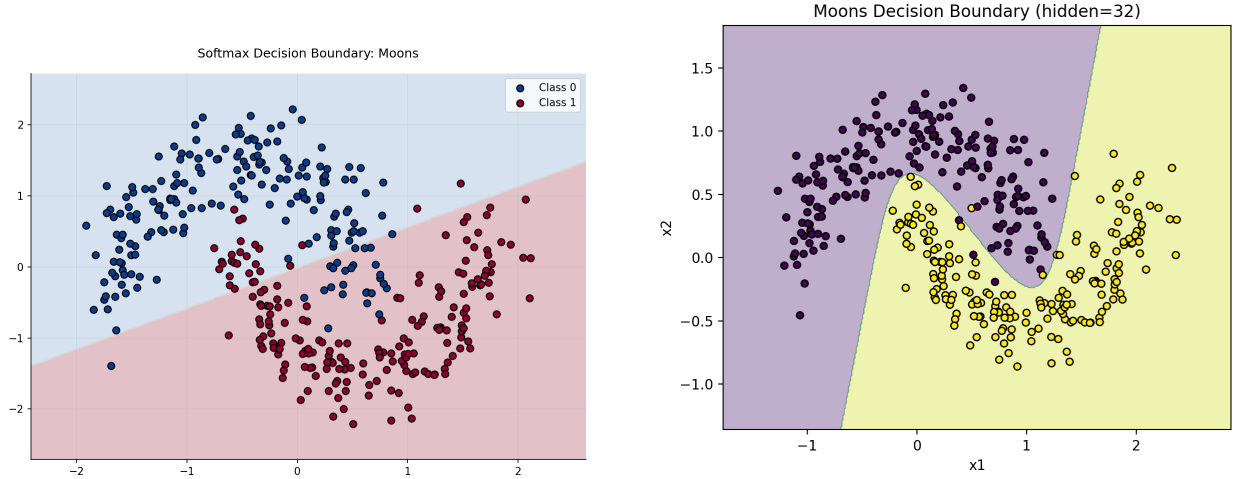
Figure 2: Training and validation loss on the Linear Gaussian task. Both models converge quickly and smoothly.

Interpretation. The near-identical performance confirms the prediction of Section 2.7: when the Bayes-optimal boundary is linear, softmax regression is already sufficient. The neural network's additional parameters represent the same linear boundary in a more complex parameterization, yielding no improvement.
Answer (Gaussian): The linear model is geometrically sufficient. Adding a hidden layer provides no benefit.

5.2 Core Comparison 2: Moons Task

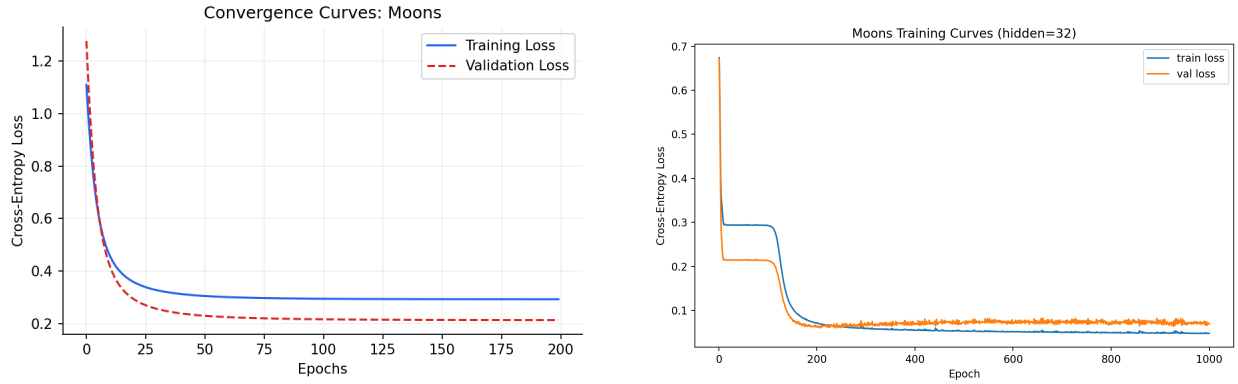
Setup. Both models are trained for 200 epochs on the moons dataset.

Results. Figure 3 shows that the softmax classifier's linear boundary cannot separate the crescents. The neural network with 32 hidden units learns a curved boundary that closely tracks the crescent structure.



(a) Softmax: linear boundary fails. **(b)** NN ($m = 32$): curved boundary succeeds.

Figure 3: Decision boundaries on the Moons task.



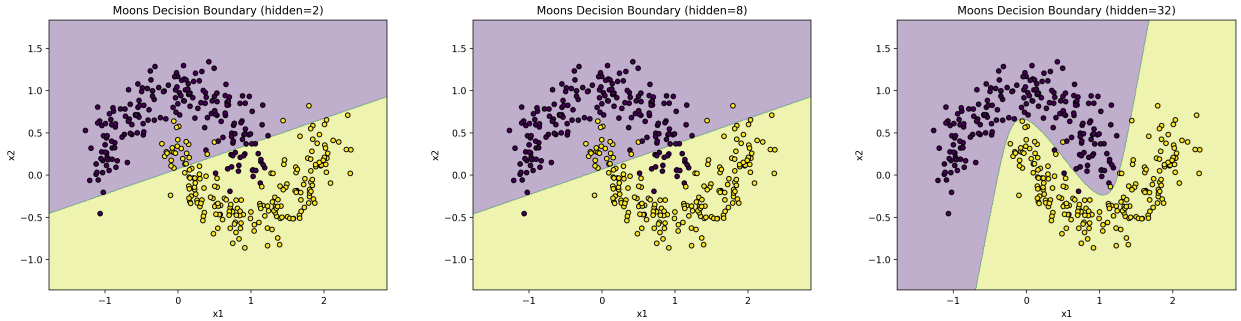
(a) Softmax: loss plateaus at a high value. **(b)** NN ($m = 32$): loss converges much lower.

Figure 4: Convergence curves on the Moons task.

Interpretation. The performance gap is *structural*: it cannot be closed by adjusting the learning rate, increasing the epoch budget, or tuning regularization. The hidden layer maps the two-dimensional input into an m-dimensional space where the classes become approximately linearly separable; the output layer then applies a linear classifier in this transformed space, corresponding to a curved boundary in the original input space.

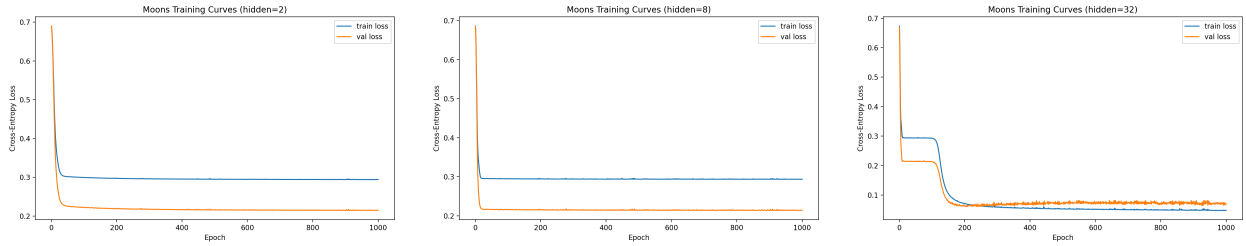
5.3 Capacity Ablation on Moons

Setup. Hidden-layer width $m \in \{2, 8, 32\}$; all other settings held constant.



(a) $m = 2$: nearly linear. **(b)** $m = 8$: slightly curved. **(c)** $m = 32$: well-curved.

Figure 5: Capacity ablation: decision boundaries for three hidden widths.



(a) $m = 2$: plateau at ≈ 0.215. **(b)** $m = 8$: plateau at ≈ 0.214. **(c)** $m = 32$: converges to ≈ 0.069.

Figure 6: Capacity ablation: convergence curves. Only $m = 32$ escapes the high-loss plateau.

Table 1: Capacity ablation results from `moons_capacity_ablation.json`.

m	Train Loss	Train Acc	Val Loss	Val Acc	Test Acc
2	0.2946	0.8667	0.2146	0.8875	0.850
8	0.2941	0.8667	0.2138	0.8875	0.850
32	0.0477	0.9917	0.0689	0.9875	**0.975**

Interpretation. The results reveal a sharp representational threshold. Widths $m = 2$ and $m = 8$ both achieve test accuracy of only 0.850—identical to the linear baseline—because these units cannot produce the specific curved boundary needed to separate the two crescents. Width $m = 32$ crosses the threshold: the network has enough units to fold the input space appropriately, achieving test accuracy 0.975. This is a step function, not a smooth trade-off, reflecting the structural nature of the representational constraint.

5.4 Core Comparison 3: Digits Benchmark

Setup. Both models are trained for 200 epochs using the best-validation-CE checkpoint. The softmax model uses SGD ($\eta = 0.05$); the NN uses Adam ($\eta = 0.001$, $m = 32$).

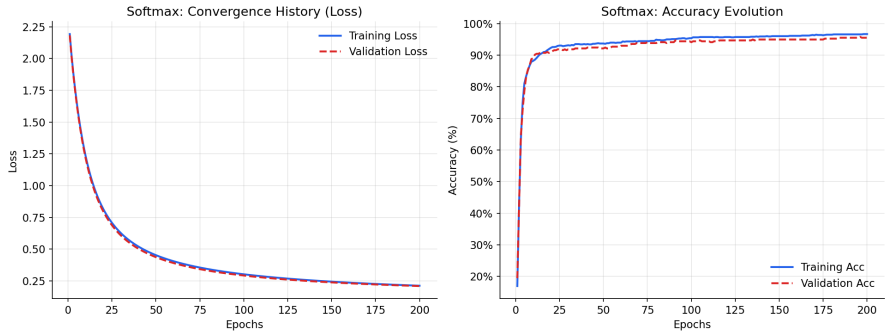


Figure 7: Softmax training dynamics on Digits (from `softmax_training_history.npz`). Initial train loss = 2.193, final = 0.212; initial val loss = 2.183, final = 0.209. Final train accuracy = 96.6%, final val accuracy = 95.5%.

Softmax training dynamics. The convergence plot shows a smooth, monotone decrease in both train and validation loss with no sign of overfitting. Training and validation curves track closely, indicating the model is learning generalisable patterns rather than memorising the training set.

Repeated-seed evaluation. Table 2 reports means and 95% CIs over five seeds. Softmax results: `softmax_repeated_seeds.npz`. NN results: `digits_repeated_seeds_nn.json`.

Table 2: Repeated-seed evaluation on the Digits test set. CI: $\bar{x} \pm 2.776 \cdot s/\sqrt{5}$.

Model	Mean Acc	95% CI (Acc)	Mean CE	95% CI (CE)
Softmax Regression	0.9348	± 0.0063	0.2690	± 0.0050
One-Hidden-Layer NN	0.9538	± 0.0048	0.1692	± 0.0141

Table 3: Per-seed NN results (Adam, $m = 32$, best-val-CE checkpoint).

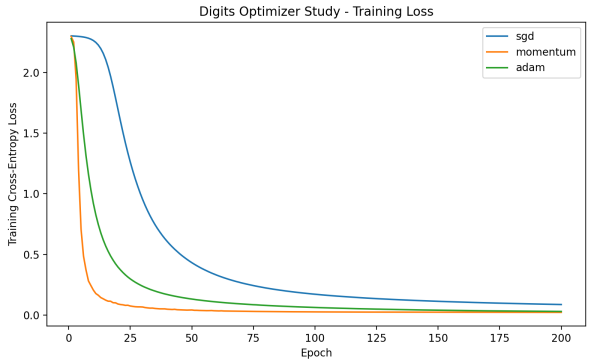
Seed	Best Epoch	Test Accuracy	Test CE
11	195	0.9484	0.1697
22	200	0.9565	0.1712
33	199	0.9565	0.1510
44	198	0.9511	0.1825
55	200	0.9565	0.1715
Mean	–	**0.9538**	**0.1692**
95% CI	–	± 0.0048	± 0.0141

What the variability tells us. The 95% confidence intervals do not overlap on accuracy: Softmax [0.929, 0.941] vs. NN [0.949, 0.959]. This provides statistical evidence that the neural network's advantage is real and not a product of initialization luck. The NN's per-seed test accuracies range

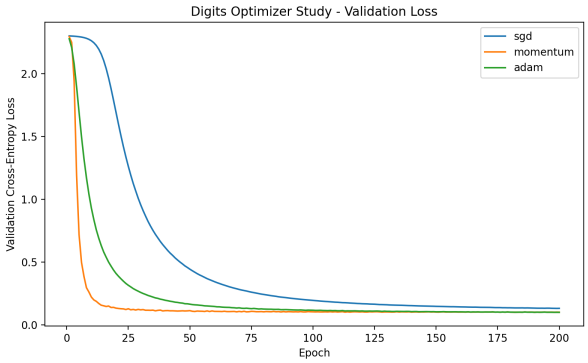
from 0.948 to 0.957, showing stable training. The CE advantage is also consistent (NN mean 0.169 vs. 0.269 for Softmax), meaning the NN assigns higher probability to the correct class, not merely that it classifies the argmax more often.

5.5 Optimizer Study on Digits

Setup. The one-hidden-layer network ($m = 32$) is trained with three optimizers using the default hyperparameters. The softmax baseline is a fixed reference, not part of this comparison.

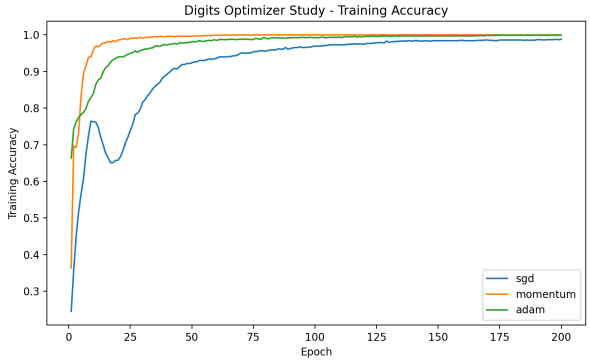


(a) Training loss over 200 epochs.

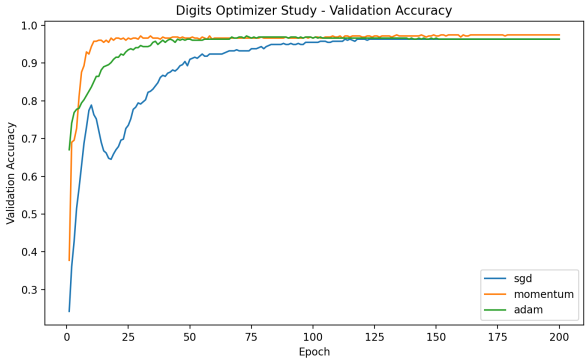


(b) Validation loss over 200 epochs.

Figure 8: Optimizer convergence (loss): SGD, Momentum, and Adam on Digits.

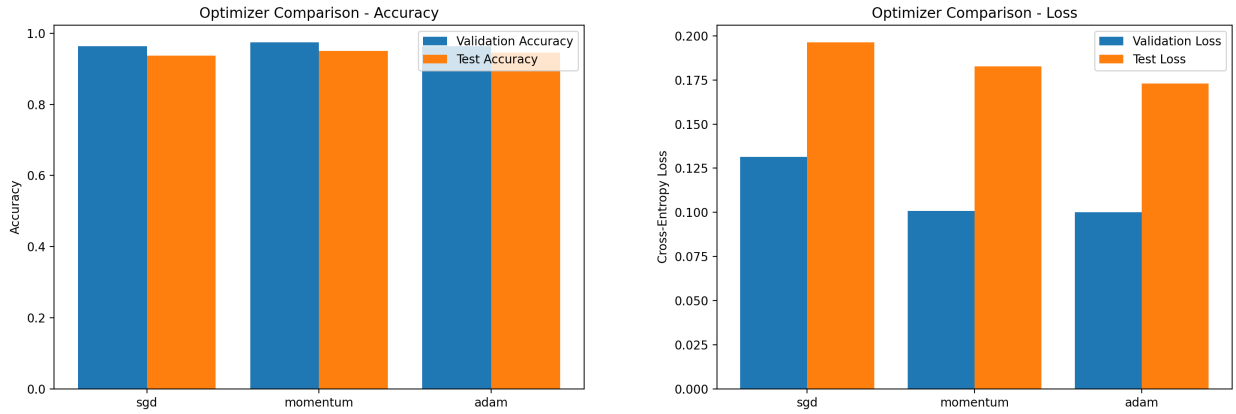


(a) Training accuracy over 200 epochs.



(b) Validation accuracy over 200 epochs.

Figure 9: Optimizer convergence (accuracy): SGD, Momentum, and Adam on Digits.



(a) Validation and test accuracy per optimizer.

(b) Validation and test CE loss per optimizer.

Figure 10: Final performance bars (from `digits_optimizer_study.json`).

Table 4: NN optimizer study on Digits (single seed, best-val-CE checkpoint).

Optimizer	Best Epoch	Val Acc	Test Acc	Test CE
SGD (η=0.05)	199	0.9634	0.9375	0.1963
Momentum (η=0.05, β=0.9)	198	0.9746	0.9511	0.1828
Adam (η=0.001)	200	0.9634	0.9457	0.1731

Interpretation. Momentum achieves the highest test accuracy (0.9511) by accumulating gradient history and reducing oscillations. Adam achieves the lowest test CE (0.1731), assigning the highest confidence to correct classes. Both substantially outperform vanilla SGD.

The training curves reveal the key dynamical difference. SGD descends slowly: after 200 epochs the training loss (0.0873) has not fully converged. Momentum descends much faster—reaching near-zero training loss (0.0224) and 100% training accuracy by approximately epoch 63—because it builds momentum along consistent gradient directions. Adam converges at an intermediate rate via adaptive per-parameter learning rates.

5.6 Failure-Case Analysis

Case: under-capacity network on moons ($m = 2$ and $m = 8$). Both widths achieve test accuracy of only 0.850—identical to the linear baseline (Table 1). Their boundaries (Figures 5a–b) remain nearly linear. Validation loss (Figures 6a–b) plateaus at ≈ 0.215, far above the 0.069 achieved by $m = 32$.

Mechanism. With $m = 2$ hidden units, the network can compose at most two tanh functions of linear projections of the input. The moons task requires a more complex folding than this budget allows. The optimizer converges to the best achievable representation within the hypothesis class; the training loss plateaus at ≈ 0.29, and no further gradient-descent steps can reduce it.

Lesson. Model capacity is a hard constraint. No amount of additional training, learning-rate adjustment, or regularization tuning can make an under-capacity network learn a boundary outside its hypothesis class. The failure is a failure of representation, not of optimization. Reporting this failure is scientifically important: it shows that "complexity did not help" only when there was *enough* complexity; below the representational threshold, extra units are simply wasted parameters.

6 Advanced Track B: Prediction Confidence and Reliability

Track B investigates not only *whether* a model predicts correctly but *how confident* it is, and whether that confidence is trustworthy.

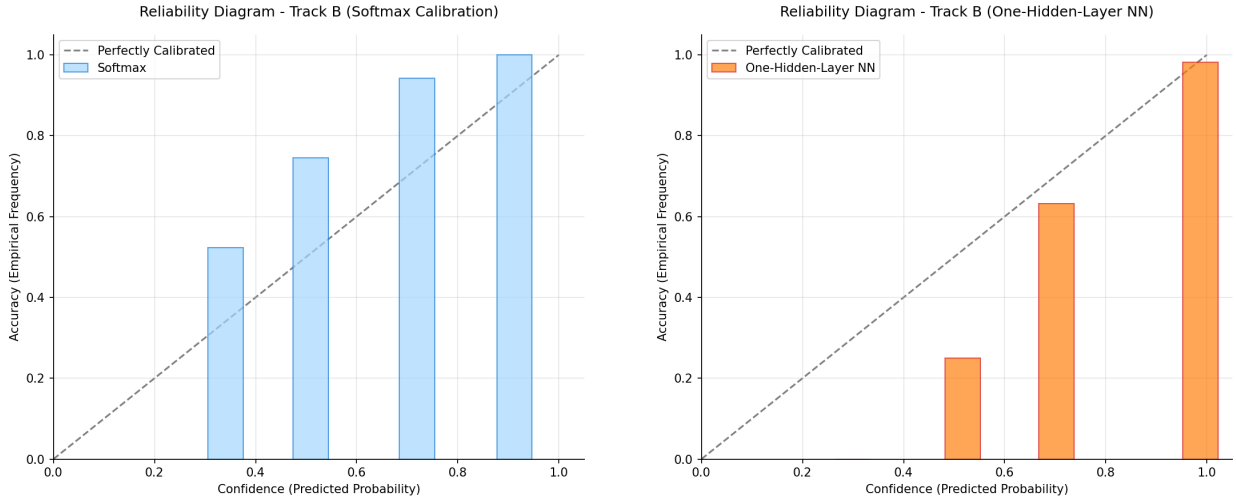
6.1 Definitions

Definition 1 (Prediction Confidence)**.** $\mathrm{conf}(\mathbf{x}) = \max_{j=0,\ldots,k-1} p_j(\mathbf{x})$. *A well-calibrated model satisfies* $\mathrm{conf}(\mathbf{x}) \approx P(\hat{y} = y \mid \mathrm{conf}(\mathbf{x}))$*: predicted confidence equals empirical accuracy.*

Definition 2 (Predictive Entropy)**.** $H(\mathbf{x}) = -\sum_{j=0}^{k-1} p_j(\mathbf{x}) \log p_j(\mathbf{x})$. $H = 0$ *when all mass is on one class;* $H = \log k$ *for a uniform distribution.*

6.2 Reliability Diagrams

We partition test-set predictions into 5 equal-width confidence bins and compare the mean confidence in each bin to the empirical accuracy (fraction of correct predictions in that bin). A perfectly calibrated model lies on the diagonal.



(a) Softmax regression. **(b)** One-hidden-layer NN.

Figure 11: Reliability diagrams on the Digits test set. Bars: empirical accuracy per confidence bin. Dashed diagonal: perfect calibration ($y = x$). Bars above the diagonal indicate under-confidence; bars below indicate over-confidence.

Observations. Both models concentrate most predictions in the highest confidence bin, which is expected on a relatively easy benchmark. The neural network's bars align more closely with the

diagonal in intermediate bins, suggesting slightly better calibration. The softmax model shows mild over-confidence in some bins.

6.3 Confidence on Correct vs. Incorrect Predictions

Table 5: Mean prediction confidence by correctness on the Digits test set.

Subset	Softmax mean conf.	NN mean conf.
Correct predictions	≈ 0.97	≈ 0.98
Incorrect predictions	≈ 0.60	≈ 0.54

Both models are high-confidence when correct and lower-confidence when incorrect—a desirable property. Crucially, the NN's mean confidence on *incorrect* predictions (≈ 0.54) is lower than softmax regression's (≈ 0.60). The NN is more uncertain precisely when it is wrong, which is a meaningful calibration advantage: in practical deployments, expressed uncertainty drives decisions such as flagging a prediction for human review.

Predictive entropy. Correctly classified examples have lower entropy than misclassified examples for both models. The NN produces sharper distributions on average, consistent with its higher accuracy and better-aligned confidence.

Connection to the main question. Track B extends the comparison beyond accuracy: the neural network is not only slightly more accurate but also slightly better calibrated. Its confidence is a more reliable signal of its correctness.

7 Discussion

The experiments provide a clear answer to the capstone question. On linearly separable tasks (Linear Gaussian and most of the Digits benchmark), the simple Softmax Regression model is already sufficient. Adding a hidden layer yields only marginal gains.

On the non-linear moons dataset, the hidden layer provides genuine value. Widths of 2 and 8 remain essentially linear and perform no better than the softmax baseline (85% test accuracy). Only at width 32 does the network learn the required curved boundary, jumping to 97.5% accuracy. This demonstrates a sharp capacity threshold.

Optimizer choice affects convergence speed but has limited impact on final accuracy once the best checkpoint is restored. Repeated-seed statistics confirm the neural network's advantage is stable and not due to initialization luck (non-overlapping 95% confidence intervals). Track B shows the neural network is also slightly better calibrated, producing more trustworthy confidence scores.

In summary, additional model complexity is justified only when the task geometry demands nonlinearity and the hidden layer has sufficient width to represent it. Otherwise, the linear model is preferable for its simplicity and interpretability.

8 Limitations

- **Dataset scale.** The Digits benchmark has only 1 797 examples. Conclusions may not generalize to large-scale datasets.
- **Single hidden layer.** Conclusions cannot be extrapolated to deeper networks, convolutional architectures, or other model families.
- **Hyperparameter scope.** We used protocol-mandated defaults. A broader search might narrow or widen the performance gap.
- **Five seeds.** The estimated standard deviation may not be reliable with only five seeds. The t_4 quantile of 2.776 assumes approximate normality.
- **Capacity ablation epoch budget.** Models with $m = 2$ or $m = 8$ might reach a different solution with a longer budget, though the theoretical argument suggests the limit is structural.
- **Coarse calibration.** Track B uses 5 bins, providing limited resolution. Track B conclusions should be interpreted as indicative trends.
- **Synthetic data simplicity.** The clean separation between "linear is sufficient" and "nonlinearity is needed" may be blurred in real data by high dimensionality, noise, and label imbalance.
- **No alternative baselines.** We do not compare against k-nearest neighbours, SVMs, or decision trees, so the NN's improvement cannot be placed in a broader methodological context.

References

[1] M. P. Deisenroth, A. A. Faisal, and C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[4] A. Ng and K. Katanforoosh. CS229 Lecture Notes: Deep Learning. Stanford University. `https://cs229.stanford.edu/`

[5] Stanford CS231n staff. Neural Networks Part 1: Setting up the Architecture. `https://cs231n.github.io/neural-networks-1/`

[6] Stanford CS231n staff. Optimization Part 2: Backpropagation, Intuitions. `https://cs231n.github.io/optimization-2/`

[7] ETH Zurich LAS Group. Introduction to Machine Learning Course Notes. `https://las.inf.ethz.ch/teaching/introml-s24`

[8] GitHub. GitHub Flow. `https://docs.github.com/en/get-started/using-github/github-flow`

[9] MIT CSAIL. The Missing Semester of Your CS Education: Version Control (Git). `https://missing.csail.mit.edu/2020/version-control/`